

Seminarium 2 del A

Typer, operatorer och uttryck

Jonas Svensson
Ulf Liedberg
Patrik Christianson

Konstanter & variabler

enum - ett alternativ till define, med möjlighet till automatisk bestämning av värden...

```
enum boolean {NO, YES}
```

och sedan...

```
if(a == NO) {  
    ... ..  
    ... ..  
}
```

För att tilldela och deklarerar:

```
int i = 12;  
char bokstaver[i];  
const double pi = 3.14;
```

Operatorer

*Aritmetiska

+, -, *, /, %

*Relationsoperationer

>, >=, <, <=

*Likhet

==, !=

*Logiska

&&, ||

*Inverterande

!

Bitvis operatorer

& - bitvis and

| - bitvis or

^ - bitvis xor

<< - vänstershift

>> - högershift

~ - komplement

Listigheter

`i = i + 1;`

kan skrivas på flera sätt:

`i++;`

`++i;`

`i+=1;`

`if(a != b)`

`q = 12;`

`else`

`q = b;`

kan också skrivas

`q = (a!=b) ? 12 : b;`

Seminarium 2 del B

Flödeskontroll

if - villkorsstyrda beslut

```
if(uttryck1)
    sats1;
else if(uttryck2)
    sats2;
else
    sats3;
```

switch

```
switch (uttryck){  
    case konstant1: satser1;  
    case konstant2: satser2;  
    default: satser3;  
}
```

Varje sats avslutas med break; annars gör den flera "case".

Ex.

```
switch (i){  
    case 0:  
        printf("i=%d", ++i);  
        break;  
    case 1:  
        printf("i=%d", --i);  
        break;  
    default:  
        printf("i=%d", i);  
        break;  
}
```


Loopar - while & do while

```
while(uttryck){  
    satser  
}
```

Ex. för att sätta i till ett tal mellan 0 och 5...

```
unsigned int i = rand();  
while(i > 5)  
    i = rand();
```

```
unsigned int i;  
do  
    i = rand();  
while(i > 5);
```

```
(unsigned int i = rand() % 5;)
```

Loopar - for

```
for (uttryck1; uttryck2; uttryck3)  
    satser
```

Ex.

```
char *monader[] = {"Januari", "Februari", "Mars",  
"April", "Maj", "Juni", "Juli", "Augusti",  
"September", "Oktober", "November",  
"December"};
```

```
for (i = 0; i < 12; i++){  
    printf("%s\n", monader[i]);  
}
```

break & continue

break

Hoppa ur loop.

continue

Hoppa till nästa iteration.

Ex.

```
int lista[] = {1, 2, 3, -4, 5, 0,4};
```

```
for (i = 0; i < 6; i++){  
    if(lista[i] < 0)  
        continue;  
    else if(lista[i]==0)  
        break;  
  
    printf("%d\n", lista[i]);  
}
```

Seminarium 2 del C

Funktioner och programstruktur

Funktioner

```
returtyp funktionsnamn (argumentdeklarationer)
{
    deklarationer

    satser

    return uttryck
}
```

Ex.

```
int mul (unsigned int a, unsigned int b)
{
    if ( a == 1)
        return b;

    return b + mul(a-1, b);
}
```

Lokala & globala variabler

Variabler har olika scope. Det innebär att en variabel deklarerad i en funktion inte kan ses av en annan funktion. Detsamma gäller variabler deklarerade i block.

En variabel som är globalt deklarerad kan ses av alla funktioner.

Ex.

```
int a = 2;
```

```
void main()  
{  
    do_things(a, 2);  
}
```

```
int do_things (int a, int b)  
{  
    return a * val(b);  
}
```

```
int val (int a)  
{  
    return a << 2;  
}
```

Static & External

En variabel kan deklarerars som static.

För en lokal variabel innebär det att dess innehåll sparas tills nästa gång.

Static på en global variabel eller funktion begränsar dess scope till filen den finns i.

Ex.

```
void counter()
{
    static int count = 0;

    printf("%d\n", count++);
}
```

Headerfiler

För att göra koden mer överblickbar bör man dela upp sitt program i flera filer.

Dessutom finns det flera standardbibliotek man kan inkludera i sitt program för att dra nytta av de funktioner de tillhandahåller.

Ex.

```
#include <stdio.h>
```

```
#include <math.h>
```

math.h tillhandahåller flera matematiska funktioner såsom kvadratroten och upphöjt till.